



Documento 7: Especificaciones de Infraestructura y DevOps

1. Arquitectura de Sistema: Monolito Modular

Para optimizar recursos y facilitar el desarrollo inicial, utilizaremos una arquitectura de **Monolito Modular** en el backend.

- **Tecnología:** NestJS (Node.js Framework).
- **Estructura:** La lógica se organiza en módulos independientes dentro del mismo proyecto:
 - : Gestión de sesiones y roles.
 - : Lógica de Stripe y procesamiento OCR.
 - : Sincronización de eventos y disponibilidad.
 - : Gestión de setlists y archivos multimedia.
- **Ventaja de Arquitecto:** Esta estructura permite que, si el tráfico aumenta, podamos extraer un módulo (ej. Finanzas) y convertirlo en un microservicio independiente sin reescribir todo el código.

2. Autenticación Centralizada: Keycloak (SSO)

Implementaremos **Keycloak** como Identity Provider (IdP) para gestionar el acceso único (Single Sign-On).

- **Función:** Los usuarios se registran una vez y pueden acceder a la App Mobile, la Plataforma Web y cualquier futura app del ecosistema RégieArt.
- **Seguridad:** Keycloak maneja tokens JWT, encriptación y protocolos estándar (OpenID Connect / OAuth2), eliminando la necesidad de guardar contraseñas en nuestra base de datos relacional.

3. Estrategia de Contenerización: Docker

Dockerizaremos los componentes críticos para asegurar la paridad entre los entornos de desarrollo y producción.

- **Backend (NestJS):** Imagen de Docker optimizada (Node-Alpine) para ejecución en Railway.
- **Frontend Web (React):** Imagen de Docker para despliegue consistente.
- **Beneficio:** "Si funciona en tu computadora, funciona en el servidor". Docker garantiza que las librerías de OCR o las dependencias de red no fallen al desplegar.

4. Ecosistema de Hosting y Costos (Estimado)

Optimizaremos el presupuesto (~5€/mes) utilizando servicios especializados:

Componente	Plataforma	Motivo Profesional
Backend API	Railway	Soporta despliegue nativo de Docker y escalabilidad vertical rápida.
Base de Datos	Supabase (Postgres)	Ofrece una instancia de PostgreSQL de alto rendimiento con capa gratuita robusta.
Frontend Web	Vercel	Optimizado para React/Next.js con CDN global integrada.
App Mobile	Expo/Stores	Despliegue nativo en Google Play y Apple App Store.

5. Automatización CI/CD: GitHub Actions

Implementaremos un pipeline de **Integración y Despliegue Continuo** para que cada a la rama principal ejecute el siguiente flujo automático:

1. **Test Stage:** Ejecución de pruebas unitarias y de integración (Jest).
2. **Quality Stage (SonarQube):** Análisis estático para detectar bugs, vulnerabilidades de seguridad y deuda técnica.
3. **Build Stage:** Construcción de las imágenes de Docker para el Backend y la Web.
4. **Deploy Stage:** * Push de la imagen al registro de Railway.
 - Despliegue automático en Vercel para el frontend.

6. Herramientas de "Arquitecto Senior" a Dominar

Para implementar este documento con éxito, estas son tus prioridades de aprendizaje:

- **Terraform:** Para definir esta infraestructura como código y poder replicarla con un comando.
- **Docker Compose:** Para levantar todo el entorno (DB, Backend, Keycloak) localmente con un solo archivo.
- **GitHub Actions Secrets:** Gestión segura de claves de API (Stripe, AWS, Keycloak).